Architectural Blueprint and Integration Logic for Autonomous Edge-Native Video Repurposing Agents (v2.0)

The deployment of an autonomous, edge-native video repurposing agent (v2.0) within a Windows 11 WSL2 Docker container represents a complex convergence of localized machine learning, hardware acceleration, and dynamic memory orchestration. Operating specifically on an NVIDIA RTX 5080 Mobile GPU equipped with 16GB of VRAM necessitates the implementation of a highly disciplined resource allocation strategy. The primary architectural constraint dictates that the system must maintain a strict peak VRAM footprint of less than 14GB. This 2GB buffer is non-negotiable; it accounts for the Windows Display Driver Model (WDDM) overhead introduced by WSL2 GPU Paravirtualization (GPU-PV), host operating system display reservations, and the inherent memory fragmentation that occurs during sequential deep learning inference tasks.

To satisfy these constraints, the architecture relies on a "Hub-and-Spoke" distributed process model. Rather than monolithic execution, a persistent Hub process coordinates the ephemeral instantiation and destruction of PyTorch-based Spokes. Each Spoke is dedicated to a singular task—such as transcription, spatial tracking, or contextual reasoning—and communicates with the Hub via a 100MB Shared CUDA Buffer. By leveraging Inter-Process Communication (IPC) handles, the system completely circumvents the Peripheral Component Interconnect Express (PCIe) bus, allowing massive multi-dimensional tensors to be passed between independent Python processes without triggering a CPU-bound host-to-device memory copy.

The successful integration of mid-2026 state-of-the-art (SOTA) models into this Hub-and-Spoke architecture requires rigorous control over the PyTorch caching allocator, precise tensor routing using Deep Learning Pack (DLPack), and seamless toggling between local Small Language Models (SLMs) and cloud-delegated Agentic Vision frameworks. This report provides an exhaustive technical breakdown of the integration logic, library dependencies, and exact code blueprints required to deploy WhisperX, YOLO26s-Pose, Qwen3-4B, and Gemini 3 Flash within this highly constrained edge environment.

Foundational Memory Orchestration and CUDA IPC Routing

Before initializing any neural network components, the underlying memory allocator must be configured to prevent the catastrophic fragmentation that typically plagues long-running computer vision pipelines. PyTorch's default caching allocator attempts to hold onto VRAM to speed up future allocations. However, in an ephemeral Spoke architecture, where vastly different models are loaded and unloaded sequentially, this behavior quickly leads to

Out-Of-Memory (OOM) exceptions even when theoretical free memory exists.

To circumvent this, the system mandates the definition of the PYTORCH_CUDA_ALLOC_CONF environment variable prior to the importation of the PyTorch library. The directive expandable_segments:True is critical; it alters the allocator's behavior to utilize CUDA virtual memory management. This allows PyTorch to dynamically map and unmap physical VRAM pages to a contiguous virtual address space, effectively returning unused memory segments to the operating system without destroying the surrounding memory pool. Furthermore, appending max_split_size_mb:512 prevents the allocator from fracturing large memory blocks. Because the architecture relies on a persistent 100MB Shared CUDA Buffer, ensuring that large, contiguous blocks remain unsplit guarantees that IPC handle generation will not fail due to a lack of contiguous physical memory addresses.

Configuration Parameter	**Allocator Function**	**Architectural Impact on 16GB RTX 5080 Mobile**
expandable_segments:True	Enables dynamic virtual memory mapping.	Prevents VRAM fragmentation across ephemeral Spoke lifecycles, maintaining the <14GB ceiling.
max_split_size_mb:512	Restricts block subdivision.	Preserves large contiguous VRAM blocks for the 100MB IPC buffer and unquantized tensor operations.

The foundation of the zero-copy Hub-and-Spoke architecture is the CUDA IPC mechanism. When the Hub process allocates the 100MB buffer, it invokes cudaIpcGetMemHandle, which returns a 64-byte stub describing the physical memory location on the GPU. This highly compact stub is then serialized and transmitted to the ephemeral Spoke process via standard Python multiprocessing queues or Unix domain sockets.[1] Upon receipt, the Spoke process invokes cudaIpcOpenMemHandle, which directly maps the physical memory into the Spoke's localized virtual address space.[1]

This methodology ensures that video frames decoded by the hardware and textual embeddings generated by the language models remain entirely resident on the RTX 5080 Mobile's High Bandwidth Memory (HBM). The implementation of this foundational routing logic requires precise interaction with PyTorch's reduction protocols to ensure the IPC handle

survives the cross-process boundary.

Python

```python
import os

# 1. Enforce strict VRAM fragmentation management for the RTX 5080 Mobile
# This must be executed prior to importing torch to alter the caching allocator.
os.environ = "expandable_segments:True,max_split_size_mb:512"

import torch
import torch.multiprocessing as mp

def initialize_persistent_hub_buffer(buffer_size_mb: int = 100) -> tuple:
    """
    Allocates a persistent, contiguous block of VRAM in the Hub process.
    Generates the 64-byte IPC handle required for zero-copy Spoke access.
    """
    # Allocate uninitialized VRAM directly on the GPU
    buffer_bytes = buffer_size_mb * 1024 * 1024
    shared_cuda_tensor = torch.empty(buffer_bytes, dtype=torch.uint8, device='cuda')

    # Extract the CUDA IPC handle
    # reduce_tensor serializes the tensor metadata and the physical memory descriptor
    ipc_handle = torch.multiprocessing.reductions.reduce_tensor(shared_cuda_tensor)

    return shared_cuda_tensor, ipc_handle

def ephemeral_spoke_ingestion_protocol(ipc_handle: tuple) -> torch.Tensor:
    """
    Executed strictly within the isolated Spoke process.
    Reconstructs the PyTorch tensor mapping to the exact physical VRAM
    without initiating a host-to-device memory transfer.
    """
    # Rebuild the tensor utilizing cudaIpcOpenMemHandle under the hood
    # The reconstruction logic unpacks the 64-byte stub and maps the virtual address space
    rebuild_function, rebuild_args = ipc_handle, ipc_handle
    spoke_tensor = rebuild_function(*rebuild_args)

    return spoke_tensor
```

This foundational layer isolates the execution contexts. If a Spoke process experiences an unrecoverable exception or memory leak, the Hub process can terminate it forcefully via the operating system without corrupting the persistent 100MB buffer or triggering a system-wide GPU panic.

Deliverable 1: Audio-Visual Alignment via WhisperX and Hardware-Accelerated AV1

The initial substantive processing phase in the video repurposing pipeline addresses the extraction of orthographic transcriptions and their alignment to the temporal axis of the video. Standard Automatic Speech Recognition (ASR) systems, including foundational versions of OpenAI's Whisper, generate timestamps at the utterance level. Because a single utterance can span multiple seconds, utilizing these raw timestamps for automated video cutting results in catastrophic drift, frequently truncating the speaker's first or last words.[4] Frame-accurate repurposing mandates millisecond-level precision.

To achieve this, the architecture integrates WhisperX, a multi-stage framework that heavily modifies the traditional ASR pipeline. WhisperX relies on Voice Activity Detection (VAD) via Pyannote-audio to explicitly identify the presence of human speech, thereby virtually eliminating hallucinations and reducing the overall processing batch size without degrading the Word Error Rate (WER).[4] For the core transcription, Distil-Whisper Large-v3 is selected. This distilled variant provides the linguistic accuracy of the massive Large-v3 model but with a drastically reduced parameter count, ensuring the Spoke process remains well within the VRAM budget.[5]

However, the critical differentiator for frame-accurate cutting is the forced alignment phase. Forced alignment utilizes a phoneme-based architecture, specifically a language-specific wav2vec2 model, to realign the transcribed text back onto the raw audio waveform.[4] By analyzing the phonetic structure of the generated text and mapping it to the acoustic features of the audio, WhisperX extracts precise millisecond boundaries for every single word.[4] Pyannote 4.0 is subsequently executed to perform speaker diarization, identifying distinct speakers and clustering the word-level segments accordingly.[4]

WhisperX Ephemeral Initialization and Forced Alignment

The Python implementation of the WhisperX Spoke must aggressively manage memory, explicitly invoking garbage collection and emptying the CUDA cache between the transcription, alignment, and diarization phases to respect the strict <14GB VRAM ceiling.

```python
```

```python
import whisperx
import gc
import torch

def execute_audio_alignment_spoke(audio_filepath: str, hf_auth_token: str) -> dict:
    """
    Ephemeral Spoke execution for WhisperX. Sequentially loads the ASR,
    Forced Alignment, and Diarization models, clearing VRAM between stages.
    """
    execution_device = "cuda"
    # Leverage the RTX 5080's Tensor Cores with half-precision floating point
    tensor_precision = "float16"

    # Stage 1: Initial Transcription via Distil-Whisper Large-v3
    # Generates accurate text but imprecise, utterance-level timestamps
    asr_pipeline = whisperx.load_model(
        "distil-large-v3",
        execution_device,
        compute_type=tensor_precision
    )
    raw_audio_waveform = whisperx.load_audio(audio_filepath)
    transcription_result = asr_pipeline.transcribe(raw_audio_waveform, batch_size=16)

    # Aggressively reclaim VRAM prior to loading the phoneme-based model
    del asr_pipeline
    gc.collect()
    torch.cuda.empty_cache()

    # Stage 2: Forced Alignment via wav2vec2
    # Maps orthographic transcripts to the acoustic signal for millisecond precision
    alignment_model, alignment_metadata = whisperx.load_align_model(
        language_code=transcription_result["language"],
        device=execution_device
    )

    aligned_segments = whisperx.align(
        transcription_result["segments"],
        alignment_model,
        alignment_metadata,
        raw_audio_waveform,
        execution_device,
```

```
    return_char_alignments=False
  )

  # Reclaim VRAM prior to Speaker Diarization
  del alignment_model
  gc.collect()
  torch.cuda.empty_cache()

  # Stage 3: Speaker Diarization via Pyannote 4.0
  # Partitions the audio stream into homogeneous speaker segments
  diarization_pipeline = whisperx.DiarizationPipeline(
    use_auth_token=hf_auth_token,
    device=execution_device
  )
  speaker_segments = diarization_pipeline(audio_filepath, min_speakers=1, max_speakers=5)

  # Project the speaker IDs onto the millisecond-aligned words
  fully_attributed_result = whisperx.assign_word_speakers(speaker_segments,
aligned_segments)

  # Final cleanup before the Spoke process terminates
  del diarization_pipeline
  gc.collect()
  torch.cuda.empty_cache()

  return fully_attributed_result
```

Data Handoff and Hardware-Accelerated AV1 Execution

Once the fully_attributed_result dictionary is generated, the agent utilizes the word-level arrays to identify the precise start and end times of the target video segment. These boundaries, which are formatted as floating-point seconds (e.g., 12.450), must be translated into hardware-accelerated FFmpeg commands to execute the physical video cut.

To maximize encoding efficiency and output fidelity, the agent leverages the AV1 codec via the NVIDIA NVENC hardware encoder present on the Blackwell-based RTX 5080 Mobile. Using the -c:v av1_nvenc parameter offloads the encoding workload from the host CPU to dedicated silicon logic on the GPU. To achieve the highest possible visual quality, the encoder is parameterized with -preset p7 (the slowest, highest quality preset) and -tune hq (high-quality tuning info).[6]

FFmpeg Parameter	Blackwell NVENC Function	Repurposing Agent Application
-c:v av1_nvenc	Invokes the hardware AV1 encoder.	Bypasses CPU encoding, drastically reducing processing time and power consumption.
-preset p7	Selects the highest quality encoding preset.	Maximizes visual fidelity for the final repurposed asset, prioritizing quality over raw encoding latency.[6]
-tune hq	Tunes the encoding pipeline for high quality.	Prevents degradation in high-motion scenes, ensuring crisp transitions.[6]
-cq:v 20	Establishes a Constant Quality rate factor.	Acts similarly to CRF, dynamically adjusting bitrate to maintain perceptual quality.[8]

For frame-accurate cutting, FFmpeg requires the precise formatting of the seek parameters. The start time (-ss) must be placed before the input flag (-i) to ensure the decoder seeks rapidly without sequentially decoding every prior frame. Furthermore, instead of an absolute end time, it is highly recommended to compute the exact duration of the clip and pass it via the -t parameter.[9] The temporal coordinates must be formatted strictly as HH:MM:SS.xxx to preserve the millisecond precision extracted from the wav2vec2 alignment.[9]

Python

```python
def generate_blackwell_av1_command(word_segment: dict, source_video_path: str, output_video_path: str) -> str:
    """
    Translates WhisperX millisecond timestamps into a frame-accurate,
    hardware-accelerated FFmpeg cutting command utilizing the NVENC AV1 encoder.
    """
    # Extract floating-point temporal boundaries from the alignment output
    start_time_float = word_segment.get("start")
```

```
    end_time_float = word_segment.get("end")

    if start_time_float is None or end_time_float is None:
        raise ValueError("Critical error: Word segment is missing wav2vec2 alignment timestamps.")

    # Calculate the precise duration of the required cut
    duration_float = end_time_float - start_time_float

    # Lambda to strictly format seconds into the HH:MM:SS.xxx FFmpeg standard
    def format_to_ffmpeg_timestamp(seconds: float) -> str:
        hours = int(seconds // 3600)
        minutes = int((seconds % 3600) // 60)
        secs = int(seconds % 60)
        milliseconds = int((seconds - int(seconds)) * 1000)
        return f"{hours:02d}:{minutes:02d}:{secs:02d}.{milliseconds:03d}"

    formatted_start = format_to_ffmpeg_timestamp(start_time_float)
    formatted_duration = format_to_ffmpeg_timestamp(duration_float)

    # Construct the highly parameterized NVENC AV1 command [6, 8, 9]
    # -ss is positioned prior to -i to enable fast seeking
    # -rc-lookahead 48 allows the encoder to analyze future frames to optimize bitrate allocation
    ffmpeg_execution_string = (
        f"ffmpeg -ss {formatted_start} -t {formatted_duration} -i {source_video_path} "
        f"-c:v av1_nvenc -preset p7 -tune hq -cq:v 20 -rc-lookahead 48 "
        f"-c:a copy -y {output_video_path}"
    )

    return ffmpeg_execution_string
```

By explicitly mapping the phoneme boundaries to the physical frame cuts via this precise syntax, the agent achieves flawless temporal sync, entirely eliminating the utterance-level drift that degrades traditional AI video editing workflows.

Deliverable 2: YOLO26s-Pose Zero-Copy Ingestion and Matrix Slicing

While WhisperX handles the temporal dimension, spatial tracking and dynamic reframing are managed by the YOLO26s-Pose object detection model. The evolution of the YOLO architecture culminated in the mid-2026 release of YOLO26, a model family fundamentally redesigned for high-efficiency edge deployment. The defining characteristic of YOLO26 is its "deployment-first philosophy," achieving end-to-end multi-task inference by permanently

removing the Non-Maximum Suppression (NMS) post-processing bottleneck and eliminating Distribution Focal Loss (DFL).[12]

The removal of NMS is a structural breakthrough. Historically, object detectors output thousands of overlapping bounding box predictions, relying on a computationally expensive CPU-bound NMS algorithm to filter redundancies based on Intersection over Union (IoU) thresholds. YOLO26 utilizes a dual assignment head that natively outputs the optimal prediction, drastically reducing computational latency and simplifying the tensor parsing logic required by the Spoke process.[12] Additionally, the introduction of the MuSGD optimizer during training ensures high accuracy on small, fast-moving objects, making the "small" variant (YOLO26s) exceptionally capable of tracking subjects across varied focal lengths.[12]

To continuously process high-definition video frames through the YOLO26s-Pose model without exhausting the VRAM, the system integrates PyNvVideoCodec and DLPack to establish a zero-copy data routing pipeline. PyNvVideoCodec interfaces directly with the NVIDIA Hardware Video Decoder (NVDEC), decoding the compressed video stream into raw NV12 or YUV444p frames entirely within the VRAM.[16] If standard PyTorch ingestion protocols are utilized, the system implicitly copies these VRAM-resident frames to system RAM (CPU) and back to the GPU to construct the continuous tensor, creating an immense PCIe bottleneck.[17]

DLPack (Deep Learning Pack) standardizes the in-memory representation of tensors. By wrapping the PyNvVideoCodec output in a DLPack structure, the Python Spoke can invoke torch.from_dlpack, seamlessly importing the physical CUDA array into the PyTorch allocator without triggering a memory copy.[16]

Zero-Copy Frame Routing

Python

```python
import torch
import PyNvVideoCodec as nvc
from ultralytics import YOLO

def initialize_yolo26_pose_spoke() -> YOLO:
    """
    Initializes the end-to-end NMS-free YOLO26s-Pose model.
    The 's' variant is chosen to balance high spatial accuracy with
    the stringent inference latency requirements of the RTX 5080.
    """
    # Loads the multi-task pose estimation weights [13, 19]
```

```
    spatial_model = YOLO("yolo26s-pose.pt")
    spatial_model.to("cuda")
    return spatial_model

def execute_zero_copy_ingestion(decoder_instance: nvc.PyNvDecoder, spatial_model: YOLO) -> list:
    """
    Decodes video frames directly into VRAM and routes them into the
    YOLO26 pipeline utilizing DLPack, completely bypassing the PCIe bus and CPU.[16, 17, 18]
    """
    batch_processing_size = 16
    # NVDEC returns frames physically residing on the GPU [18]
    hardware_decoded_frames = decoder_instance.get_batch_frames(batch_processing_size)

    if len(hardware_decoded_frames) == 0:
        return

    zero_copy_tensor_batch =
    for raw_frame in hardware_decoded_frames:
        # Convert the proprietary NVDEC frame into a PyTorch tensor via DLPack
        # This is an O(1) operation mapping the virtual address pointer.
        # Default shape is for RGB data.
        frame_tensor = torch.from_dlpack(raw_frame)

        # YOLO26 requires channel-first layout
        # Permute manipulates the tensor stride, not the physical memory, maintaining zero-copy.
        frame_tensor = frame_tensor.permute(2, 0, 1).float() / 255.0

        # Add the batch dimension:
        frame_tensor = frame_tensor.unsqueeze(0)
        zero_copy_tensor_batch.append(frame_tensor)

    # Concatenate the list into a single batch tensor for efficient parallel inference [16]
    final_batch_tensor = torch.cat(zero_copy_tensor_batch, dim=0)

    # Execute NMS-free inference leveraging native FP16 Tensor Core acceleration [12, 14]
    prediction_results = spatial_model.predict(final_batch_tensor, half=True, verbose=False)

    return prediction_results
```

Virtual Slicing of the Unified Output Matrix

Because YOLO26 is a unified multi-task architecture, its prediction output intrinsically links the object detection bounding box with its corresponding pose estimation keypoints.[12] The raw

tensor output requires precise mathematical slicing to separate the distinct components required for the downstream tracking algorithms.

The downstream video tracking mechanism utilizes a dual-layered approach:

1. **ByteTrack Association:** Requires the full-body bounding box formatted as (N, 6) containing [x1, y1, x2, y2, confidence, class]. ByteTrack utilizes Intersection over Union (IoU) and appearance heuristics to maintain subject identity across frames.
2. **Kalman Filter Framing:** Relies purely on the facial landmarks. Attempting to center a reframed 9:16 vertical video on the full-body bounding box results in unacceptable visual jitter as the subject moves their arms or legs. Extracting the facial landmarks provides a highly stable center-of-mass for the Pan-Tilt-Zoom (PTZ) framing logic.

The YOLO26 pose model is trained on the standard COCO keypoints dataset, which predicts 17 distinct anatomical joints.[19] The keypoint matrix is a three-dimensional tensor of shape (N, 17, 3), representing N detected persons, 17 keypoints, and the 3 values of [x, y, visibility].[19]

Keypoint Index	Anatomical Joint Mapping	Slicing Requirement
Index 0	**Nose**	**Targeted for extraction**
Index 1	**Left Eye**	**Targeted for extraction**
Index 2	**Right Eye**	**Targeted for extraction**
Index 3	**Left Ear**	**Targeted for extraction**
Index 4	**Right Ear**	**Targeted for extraction**
Index 5 - 16	Shoulders, Elbows, Wrists, Hips, Knees, Ankles	Discarded for PTZ framing calculations [19]

According to the COCO standardization, the facial landmarks exclusively occupy the first five indices (0 through 4) of the keypoint dimension.[19] Virtual tensor slicing allows the Spoke process to isolate these coordinates without explicitly allocating new memory matrices.

Python

```python
def extract_tracking_matrices(yolo_prediction_result) -> tuple:
    """
    Extracts the isolated full-body bounding box and the 5-point facial landmark
    matrix from the unified YOLO26s-Pose tensor via virtual slicing.
    """
    # Verify the presence of detected subjects to avoid indexing errors
    if yolo_prediction_result.boxes is None or yolo_prediction_result.keypoints is None:
        return None, None

    # 1. Isolate the Bounding Box array for ByteTrack ingestion
    # Shape: (N, 6) containing the localized coordinate boundaries and confidence metrics [21, 22]
    bbox_tracking_tensor = yolo_prediction_result.boxes.xyxy

    # 2. Extract the unified 17-point Keypoint Tensor
    # Shape: (N, 17, 3) representing N persons, 17 structural joints, and [x, y, visible] data
    full_anatomical_keypoints = yolo_prediction_result.keypoints.data

    # Execute virtual slicing to isolate the facial center-of-mass
    # Matrix Slice logic:
    # Slices Indices: 0 (Nose), 1 (L-Eye), 2 (R-Eye), 3 (L-Ear), 4 (R-Ear)
    facial_landmark_tensor = full_anatomical_keypoints[:, 0:5, :]

    # facial_landmark_tensor now represents an (N, 5, 3) matrix mapping directly
    # to the visual center of the subject's face, ideal for the Kalman Filter calculation.
    return bbox_tracking_tensor, facial_landmark_tensor
```

This strict adherence to virtual slicing guarantees that the operations do not fracture the VRAM pool, maintaining absolute compliance with the expandable_segments methodology outlined in the architectural foundation.

Deliverable 3: Ephemeral Contextual Reasoning via Qwen3-4B (INT4)

While spatial algorithms excel at localization, the autonomous agent requires advanced semantic reasoning to determine the contextual value of a video segment. The agent must continuously evaluate the metadata generated by the prior stages to identify high-retention "hooks" (e.g., sudden emotional shifts, complex thematic dialogue, or visual novelty). Depending entirely on cloud-based APIs for this continuous, frame-by-frame evaluation would incur prohibitive latency and excessive cost. Therefore, the pipeline relies on the local deployment of a Small Language Model (SLM).

The Qwen3-4B model represents the mid-2026 standard for edge reasoning, incorporating

advanced Mixture of Experts (MoE) optimizations and a highly sophisticated instruction-following framework.[23] However, loading a 4-billion parameter model in FP16 precision requires approximately 8GB of VRAM, which, when combined with the 100MB IPC buffer, the WSL2 WDDM overhead, and the transient memory spikes of the Spoke processes, endangers the 14GB ceiling. Consequently, the Qwen3-4B model must be loaded using INT4 quantization (via AWQ or GPTQ), compressing the weights to 4-bit integers and reducing the baseline memory footprint to a highly manageable ~2.5GB to 3GB.[23]

To maximize inference speed and control memory allocation within the ephemeral Python Spoke, **vLLM** is the optimal inference engine.[27] Utilizing native Hugging Face transformers with bitsandbytes lacks the sophisticated PagedAttention mechanism integral to vLLM. PagedAttention dynamically manages the Key-Value (KV) cache by mapping non-contiguous physical memory blocks to contiguous virtual blocks, drastically reducing VRAM waste and allowing the developer to set a strict upper bound on GPU memory utilization. SGLang offers similar performance, but vLLM provides a highly explicit and mature configuration syntax for toggling reasoning paradigms dynamically.[28]

Ephemeral Loading and the Dual-Mode Cognitive Toggle

A defining innovation of the Qwen3 architecture is its integration of a latent "Thinking" state, capable of executing complex Chain-of-Thought (CoT) reasoning similar to the DeepSeek-R1 framework.[25] However, this advanced reasoning introduces a critical bottleneck: the Time Between Tokens (TBT) and Time To First Visible Token (TTFVT) increase dramatically as the model generates the hidden <think> blocks.[27]

The repurposing agent operates under dual cognitive requirements. During standard operations, it performs rapid metadata scanning and binary classification (e.g., "Does this frame contain a transition?"). This task requires near-instantaneous output. Conversely, when a potential hook is flagged, the agent must perform deep thematic analysis to determine the optimal cutting strategy. Therefore, the Spoke process must dynamically toggle the model between a low-latency "Non-Thinking" state and a deep "Thinking" state using vLLM's enable_thinking configuration.[28]

Cognitive State	enable_thinking Flag	Primary Application	Inference Characteristics
Non-Thinking	False	Rapid heuristic scanning and continuous metadata	Sub-second latency, deterministic sampling (temperature=0.0),

		classification.	minimal TBT.[27]
Thinking	True	Complex narrative evaluation and thematic cut-point selection.	High latency, extensive CoT generation, elevated sampling variability (temperature=0.6).[28]

The implementation leverages vLLM's chat_template_kwargs to execute this hard switch dynamically on a per-request basis without reloading the model weights.[28]

Python

```python
from vllm import LLM, SamplingParams
import torch
import gc

def initialize_ephemeral_qwen3_spoke() -> LLM:
    """
    Initializes Qwen3-4B utilizing INT4 AWQ quantization via the vLLM engine.
    Configured specifically for aggressive VRAM management.
    """
    # The LLM engine is initialized with parameters to restrict footprint [26, 28]
    # gpu_memory_utilization establishes a strict cap, ensuring the KV cache
    # does not uncontrollably expand and breach the 14GB limit.
    # enable_reasoning prepares the backend to parse DeepSeek-R1 style output [29]
    reasoning_engine = LLM(
        model="Qwen/Qwen3-4B-Instruct-AWQ",
        quantization="awq",
        tensor_parallel_size=1,
        gpu_memory_utilization=0.35,       # Caps allocation to roughly 5.6GB of the 16GB total [26]
        enforce_eager=True,                # Disables CUDA graph caching to conserve VRAM overhead
        enable_reasoning=True              # Initializes the reasoning parser pipeline
    )
    return reasoning_engine

def execute_dual_mode_reasoning(
    reasoning_engine: LLM,
```

```
    contextual_prompt: str,
    trigger_deep_thought: bool
) -> str:
    """
    Executes inference dynamically, explicitly toggling the Chain-of-Thought
    cognitive graph based on the complexity of the analytical task.
    """
    conversation_history =

    if trigger_deep_thought:
        # Thinking Mode Configuration
        # Elevates temperature to encourage diverse exploratory logic.
        # Expands max_tokens to accommodate the extensive hidden <think> trajectory.[29]
        inference_params = SamplingParams(
            temperature=0.6,
            top_p=0.95,
            max_tokens=4096
        )
        # Apply the hard switch to force CoT engagement
        template_kwargs = {"enable_thinking": True}
    else:
        # Non-Thinking Mode Configuration [26, 28]
        # Enforces absolute determinism for rapid, predictable binary outputs.
        # Constrains max_tokens to prevent hallucinated narrative drift.
        inference_params = SamplingParams(
            temperature=0.0,
            top_p=1.0,
            max_tokens=256
        )
        # Apply the hard switch to bypass CoT entirely, minimizing TBT
        template_kwargs = {"enable_thinking": False}

    # Execute the request utilizing vLLM's internal chat templating API
    generation_output = reasoning_engine.chat(
        messages=conversation_history,
        sampling_params=inference_params,
        chat_template_kwargs=template_kwargs
    )

    # Extract the parsed visible tokens, excluding the internal <think> data
    final_textual_decision = generation_output.outputs.text
    return final_textual_decision
```

```python
def terminate_qwen3_spoke(reasoning_engine: LLM):
    """
    Executes a structured termination of the vLLM instance.
    Forcibly unmaps the PagedAttention blocks and weight matrices,
    returning the VRAM to the OS via the expandable_segments mechanism.
    """
    del reasoning_engine
    gc.collect()
    torch.cuda.empty_cache()
```

This deliberate toggling mechanism empowers the agent to process thousands of frames heuristically in seconds, reserving the computationally intensive, high-latency reasoning exclusively for the moments mathematically identified as optimal narrative transitions.

Deliverable 4: Cloud-Delegated Agentic Vision via Gemini 3 Flash

While the Qwen3-4B model handles text and metadata reasoning, and YOLO26s-Pose handles geometric localization, the agent frequently encounters complex spatial scenarios requiring profound visual mathematics. Examples include counting distinct structural elements within a crowded composition, performing Optical Character Recognition (OCR) on obscured typography, or evaluating the microscopic brushstrokes of an underlying image tensor.[30]

Attempting to run a 30B+ parameter Vision-Language Model (VLM) locally to achieve this level of analysis is fundamentally impossible on a 16GB RTX 5080 Mobile. Furthermore, standard multimodal models process images by breaking them down into static patches, frequently losing high-frequency details and "hallucinating" spatial reasoning (e.g., confidently stating there are 12 boxes in an image when 15 are physically present).[30]

To overcome this, the architecture implements a strategic API offload utilizing Google's Gemini 3 Flash and its "Agentic Vision" framework. Agentic Vision transcends the limitations of static patch-based inference by employing a deterministic Think-Act-Observe loop.[30] When queried, the model does not guess; it autonomously writes and executes Python code (frequently utilizing libraries like OpenCV) within a secure backend sandbox to mathematically crop, zoom, and analyze the image, verifying its findings before generating a response.[30]

Constructing the Agentic Vision API Execution

Integrating this cloud-delegated Spoke requires the google-genai Python SDK (v1.51.0 or later).[32] Because API calls introduce network latency and per-token costs, the request must be tightly parameterized using the types.GenerateContentConfig object to maximize processing speed while retaining the code execution capability.[30]

Two specific configuration parameters are mandatory for the video repurposing agent:

1. **Thinking Level (thinking_level=MINIMAL):** By default, Gemini 3 utilizes dynamic reasoning that can slow down responses. Because the Agentic Vision tool inherently provides deterministic code-based reasoning, the overarching textual CoT can be suppressed by setting the thinking level to MINIMAL (or LOW).[30] This preserves the sandbox logic while drastically lowering latency.
2. **Media Resolution (media_resolution=HIGH):** The code execution backend requires high-density pixel data to execute its zoom and crop algorithms effectively. Passing MEDIA_RESOLUTION_HIGH guarantees the image tensor is uploaded without lossy downscaling, preventing the target data from becoming an "inaccessible blur" for the backend Python script.[30]

Python

```python
import os
from google import genai
from google.genai import types

def execute_agentic_vision_delegation_spoke(raw_image_bytes: bytes, spatial_directive: str) -> str:
    """
    Offloads highly complex spatial mathematics to the Gemini 3 Flash API.
    Utilizes the Agentic Vision framework to execute deterministic OpenCV
    analysis on the Google backend, entirely bypassing the local RTX 5080 GPU.
    """
    # Initialize the GenAI SDK client [30, 34, 35]
    # Requires the GOOGLE_API_KEY environment variable to be configured securely
    cloud_client = genai.Client()

    # Translate the raw in-memory image bytes into a multimodal Part object [32, 35]
    # This prevents the IO overhead of writing temporary image files to the WSL2 virtual disk
    multimodal_image_part = types.Part.from_bytes(
        data=raw_image_bytes,
        mime_type="image/jpeg"
    )

    # Construct the highly parameterized GenerateContentConfig object
    request_configuration = types.GenerateContentConfig(

        # 1. Enforce high-density pixel upload
```

```
        # Crucial for allowing the backend Python script to crop and magnify granular details
        media_resolution=types.MediaResolution.MEDIA_RESOLUTION_HIGH,

        # 2. Constrain the textual reasoning verbosity [30, 34]
        # MINIMAL reduces network latency and token output while preserving tool execution logic
        thinking_config=types.ThinkingConfig(
            thinking_level=types.ThinkingLevel.MINIMAL
        ),

        # 3. Enable the Deterministic Code Execution Sandbox [31, 35]
        # Instantiates the Think-Act-Observe loop, allowing Gemini to write and
        # execute Python scripts against the image tensor to verify its analysis.
        tools=
    )

    # Transmit the request to the global endpoint [33, 35]
    api_response = cloud_client.models.generate_content(
        model="gemini-3-flash-preview",
        contents=[multimodal_image_part, spatial_directive],
        config=request_configuration
    )

    return api_response.text
```

The architectural synergy of this delegation is profound. During the asynchronous network wait state required by the Gemini API, the local RTX 5080 Mobile is entirely idle. Its VRAM remains unburdened, allowing the PyTorch allocator's expandable_segments mechanism to defragment memory or aggressively pre-allocate the 100MB IPC buffer for the subsequent local processing Spoke. This prevents the system from locking up during intense reasoning phases and guarantees absolute stability for the autonomous agent.

Through the meticulous orchestration of CUDA IPC memory management, the temporal precision of WhisperX phoneme mapping, the zero-copy DLPack ingestion of YOLO26 spatial tracking, the strategic toggling of the Qwen3 reasoning graph, and the offloading of complex vision mathematics via the Agentic Vision framework, the edge-native video repurposing agent successfully operates a suite of mid-2026 SOTA models. This architecture fundamentally proves that severe hardware constraints do not preclude advanced autonomous capabilities when software integration is engineered with uncompromising precision.

Works cited

1. One Kernel for All Your GPUs - Hazy Research, accessed on March 7, 2026, https://hazyresearch.stanford.edu/blog/2025-09-22-pgl

2. ParallelKittens: Systematic and Practical Simplification of Multi-GPU AI Kernels - arXiv, accessed on March 7, 2026, https://arxiv.org/html/2511.13940v1
3. StreamBox: A Lightweight GPU SandBox for Serverless Inference Workflow - USENIX, accessed on March 7, 2026, https://www.usenix.org/system/files/atc24-wu-hao.pdf
4. m-bain/whisperX: WhisperX: Automatic Speech ... - GitHub, accessed on March 7, 2026, https://github.com/m-bain/whisperX
5. Whisper and Pyannote: The Ultimate Solution for Speech Transcription, accessed on March 7, 2026, https://scalastic.io/en/whisper-pyannote-ultimate-speech-transcription/
6. AV1 NVENC Config for FFMPEG - GitHub Gist, accessed on March 7, 2026, https://gist.github.com/Nicklas373/6cf92b3594561bff326697a93ffb2e5b
7. Let's explore av1_nvenc together : r/AV1 - Reddit, accessed on March 7, 2026, https://www.reddit.com/r/AV1/comments/1011ujq/lets_explore_av1_nvenc_together/
8. Nvenc AV1 output quality - General - Topaz Community, accessed on March 7, 2026, https://community.topazlabs.com/t/nvenc-av1-output-quality/47683
9. ffmpeg video editing command in milliseconds timestamp - Stack Overflow, accessed on March 7, 2026, https://stackoverflow.com/questions/23171937/ffmpeg-video-editing-command-in-milliseconds-timestamp
10. Using ffmpeg to cut videos with more precision than key frames allow, accessed on March 7, 2026, https://video.stackexchange.com/questions/16750/using-ffmpeg-to-cut-videos-with-more-precision-than-key-frames-allow
11. URGENT NEED PLEASE ANSWER, BE NICE : HOW TO accurately to trim an mp4 by the millisecond (possibly with reencoding and frame cutting)? And why some editors can cut MP4s accurately while ffmpeg can't? - Reddit, accessed on March 7, 2026, https://www.reddit.com/r/ffmpeg/comments/129h1rs/urgent_need_please_answer_be_nice_how_to/
12. YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection - arXiv.org, accessed on March 7, 2026, https://arxiv.org/html/2509.25164v4
13. Ultralytics YOLO26, accessed on March 7, 2026, https://docs.ultralytics.com/models/yolo26/
14. Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8, and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition - arXiv.org, accessed on March 7, 2026, https://arxiv.org/html/2510.09653v2
15. YOLO26: A Comprehensive Architecture Overview and Key Improvements - arXiv.org, accessed on March 7, 2026, https://arxiv.org/pdf/2602.14582
16. PyNvVideoCodec API - NVIDIA Documentation, accessed on March 7, 2026, https://docs.nvidia.com/video-technologies/pynvvideocodec/pdf/PyNvVideoCodec_API_ProgGuide.pdf
17. [RFC]: Enabling Zero-Copy Video with PyNvVideoCodec and IPC · Issue #30839 -

GitHub, accessed on March 7, 2026, https://github.com/vllm-project/vllm/issues/30839
18. PyNvVideoCodec API Programming Guide - NVIDIA Docs, accessed on March 7, 2026, https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-prog-guide/index.html
19. Pose Estimation - Ultralytics YOLO Docs, accessed on March 7, 2026, https://docs.ultralytics.com/tasks/pose/
20. What Is Pose Estimation? Keypoint Detection Explained [2026] | Datature Blog, accessed on March 7, 2026, https://datature.io/blog/what-is-pose-estimation-keypoint-detection-explained-2026
21. Reference for ultralytics/engine/results.py, accessed on March 7, 2026, https://docs.ultralytics.com/reference/engine/results/
22. Qwen3-Next-80B-A3B vs gpt-oss-120b : r/LocalLLaMA - Reddit, accessed on March 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1p9nckz/qwen3next80ba3b_vs_gptoss120b/
23. Qwen3 is the large language model series developed by Qwen team, Alibaba Cloud. - GitHub, accessed on March 7, 2026, https://github.com/QwenLM/Qwen3
24. Qwen3 is all you need. Hello everyone, I'm Lao Zhang, an AI... | by zhangthinkin | Medium, accessed on March 7, 2026, https://medium.com/@venn5708/hello-everyone-im-lao-zhang-an-ai-learning-enthusiast-8f0e9c3dffc2
25. Integrate and Deploy Tongyi Qwen3 Models into Production Applications with NVIDIA, accessed on March 7, 2026, https://developer.nvidia.com/blog/integrate-and-deploy-tongyi-qwen3-models-into-production-applications-with-nvidia/
26. Reasoning Language Model Inference Serving Unveiled: An Empirical Study - arXiv, accessed on March 7, 2026, https://arxiv.org/html/2510.18672v1
27. vLLM - Qwen, accessed on March 7, 2026, https://qwen.readthedocs.io/en/latest/deployment/vllm.html
28. Qwen/Qwen3-32B - Hugging Face, accessed on March 7, 2026, https://huggingface.co/Qwen/Qwen3-32B
29. Gemini 3 Agentic Vision: Moving Beyond Simple Image Prompting - DEV Community, accessed on March 7, 2026, https://dev.to/railsstudent/building-a-digital-docent-master-agentic-vision-with-gemini-3-20i1
30. Build an Autonomous Supply Chain with Gemini 3 Flash & AlloyDB AI - Google Codelabs, accessed on March 7, 2026, https://codelabs.developers.google.com/visual-commerce-gemini-3-alloydb
31. Turning Handwritten Plans into Strategic Intelligence with Gemini 3 | by #TheGenAIGirl — code, community, and GenAI. | Google Cloud - Medium, accessed on March 7, 2026, https://medium.com/google-cloud/turning-handwritten-plans-into-strategic-intell

igence-with-gemini-3-70d57222542d
32. Get started with Gemini 3 | Generative AI on Vertex AI - Google Cloud Documentation, accessed on March 7, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/start/get-started-with-gemini-3
33. Detecting and Editing Visual Objects with Gemini | Towards Data Science, accessed on March 7, 2026, https://towardsdatascience.com/detecting-and-editing-visual-objects-with-gemini/